

Probabilistic Modeling:

1.1 LMs: Distrib. over Strings

Def: An Alphabet Σ/Δ is a finite non-empty set. Elements $a, b, c \in \Sigma$ are called symbols / letters.

Def: A string over Σ is any finite seq of letters: $x = x_1 \dots x_n$.

Def: The Kleene Star Σ^* is def as: $\Sigma^* = \bigcup_{n=0}^{\infty} \Sigma^n$ where $\Sigma^0 = \{\epsilon\}$, $\Sigma^1 = \Sigma$, $\Sigma^2 = \Sigma \Sigma$, etc.

We call Σ^* the Kleene closure of Σ . **Def:** Σ^+ is $\Sigma^* \setminus \{\epsilon\}$.

Def: A language L is a subset of Σ^* .

Def: A LLM is a (discrete) prob. p_{un} over Σ^* .

Def: The weighted lang. of p_{un} is def as $\langle p_{\text{un}}, p_{\text{un}}(x) \rangle_{x \in \Sigma^*}$.

2.2 Global & Local Normalization

Def: Let p_{un} be a prob. p_{un} over Σ^* . A globally norm. model (GNN) is defined as:

$$p_{\text{un}}(x) = \frac{\exp(-\sum_{i=1}^n p_{\text{un}}(x_i))}{Z}$$

Rem: ϕ only depends on prev. scores, entire seq of once prob. distr. over Σ directly. ϕ is difficult to compute.

Thm: Any normalizable E-fct (Z_{un}) induces a LLM over Σ^* .

Proof: $Z_{\text{un}}(x) = \sum_{y \in \Sigma^*} \exp(-\sum_{i=1}^n p_{\text{un}}(x_i)) = \sum_{y \in \Sigma^*} \exp(-\sum_{i=1}^n p_{\text{un}}(x_i))$

Def: A seq model (SM) over Σ is def as a set of cond. prob. distr. $p_{\text{un}}(y|x)$ for $y \in \Sigma$, $x \in \Sigma^*$.

Def: Let p_{un} be a SM over Σ . A locally normalized LLM (LNN) over Σ is def as:

$$p_{\text{un}}(x) = \prod_{i=1}^n p_{\text{un}}(x_i | x_{1:i-1})$$

Thm: If $p_{\text{un}}(x) = \prod_{i=1}^n p_{\text{un}}(x_i | x_{1:i-1})$ then $p_{\text{un}}(x) = \prod_{i=1}^n p_{\text{un}}(x_i | x_{1:i-1})$.

Def: A LLM is light if $Z_{\text{un}}(x) = 1$. $\Leftrightarrow p_{\text{un}}(\Sigma^*) = 1$.

Thm: SMs (LNN) may place prob. on seq. of seqs.

Thm: LNN model a series of cond. prob. distr. $p_{\text{un}}(y|x)$.

Thm: SMs (LNN) may not be LMs (i.e. can't be used as model).

Def: Prefix prob. $\pi(x) = \sum_{y \in \Sigma^*} p_{\text{un}}(xy)$ = prob. that x is a prefix of any string $y \in \Sigma^*$.

Thm: Let p_{un} be LLM. Then π is a locally normal LLM $p_{\text{un}}(x) = \prod_{i=1}^n p_{\text{un}}(x_i | x_{1:i-1})$.

Proof: $p_{\text{un}}(xy) = \prod_{i=1}^n p_{\text{un}}(x_i | x_{1:i-1}) \prod_{j=1}^m p_{\text{un}}(y_j | x_{1:n+m-1})$

Case 2: $\pi(x) = 0$: there must $\exists i \in \{1, \dots, n\}$ s.t. $p_{\text{un}}(x_i) = 0$.

Rem: Any LLM prob. can be converted into a locally normal. seq. model. But, converse is NOT true.

2.3 Tightness of LMs

Def: A seq model is a probability space over $\Sigma^* \cup \Sigma^{\infty}$.

Def: A LLM is a prob. space over Σ^* s.t. $p_{\text{un}}(\Sigma^*) = 1$.

Remarks: A LNN is light iff $\pi(\Sigma^*) = 1$. $\Leftrightarrow p_{\text{un}}(\Sigma^*) = 1$.

Def: A LLM is tight if $\pi(\Sigma^*) = 1$.

Def: A LLM is light if $\pi(\Sigma^*) = 1$.

Def: A LLM is tight if $\pi(\Sigma^*) = 1$.

Def: A LLM is light if $\pi(\Sigma^*) = 1$.

Def: A LLM is tight if $\pi(\Sigma^*) = 1$.

Def: A LLM is light if $\pi(\Sigma^*) = 1$.

Def: A LLM is tight if $\pi(\Sigma^*) = 1$.

Def: A LLM is light if $\pi(\Sigma^*) = 1$.

Def: A LLM is tight if $\pi(\Sigma^*) = 1$.

Def: A LLM is light if $\pi(\Sigma^*) = 1$.

Def: A LLM is tight if $\pi(\Sigma^*) = 1$.

Def: A LLM is light if $\pi(\Sigma^*) = 1$.

Def: A LLM is tight if $\pi(\Sigma^*) = 1$.

Def: A LLM is light if $\pi(\Sigma^*) = 1$.

Def: A LLM is tight if $\pi(\Sigma^*) = 1$.

Def: A LLM is light if $\pi(\Sigma^*) = 1$.

Def: A LLM is tight if $\pi(\Sigma^*) = 1$.

Def: A LLM is light if $\pi(\Sigma^*) = 1$.

Def: A LLM is tight if $\pi(\Sigma^*) = 1$.

Def: A LLM is light if $\pi(\Sigma^*) = 1$.

2. Modeling Foundations

2.1 Representation-based LMs

In Rep-based LMs, we param a LLM through SMs $p_{\text{un}}(y|x)$.

Def: A context enc. fct. $\text{enc}(\cdot): \Sigma^* \rightarrow \mathbb{R}^d$ is a rep.

Def: A string $x \in \Sigma^*$. Ref. to $\text{enc}(x)$ as enc. of x .

Def: The encoding matrix $E \in \mathbb{R}^{|\Sigma| \times d}$ is def as the matrix embedding of symbols $y \in \Sigma$ where S is a finite vocabul.

Def: The embedding dimension d is the embedding dimension.

Prop: Each entry of the vec. $E \cdot \text{enc}(x) \in \mathbb{R}^d$ correspond to the similarities of a particular symbol y to the context x .

Def: Transform $E \cdot \text{enc}(x)$ into prob. distr. by using proj. fct.

Def: Prob. simplex $\Delta^{d-1} = \{x \in \mathbb{R}^d | x_i \geq 0, \sum_{i=1}^d x_i = 1\}$.

Def: Prob. fct. $f: \mathbb{R}^d \rightarrow \Delta^{d-1}$.

Def: Prob. fct. $f: \mathbb{R}^d \rightarrow \Delta^{d-1}$.

Def: Softmax: $\text{softmax}(x) = \frac{\exp(x_i)}{\sum_{j=1}^d \exp(x_j)}$.

Def: Softmax: $\text{softmax}(x) = \frac{\exp(x_i)}{\sum_{j=1}^d \exp(x_j)}$.

Def: Softmax: $\text{softmax}(x) = \frac{\exp(x_i)}{\sum_{j=1}^d \exp(x_j)}$.

Def: Softmax: $\text{softmax}(x) = \frac{\exp(x_i)}{\sum_{j=1}^d \exp(x_j)}$.

Def: Softmax: $\text{softmax}(x) = \frac{\exp(x_i)}{\sum_{j=1}^d \exp(x_j)}$.

Def: Softmax: $\text{softmax}(x) = \frac{\exp(x_i)}{\sum_{j=1}^d \exp(x_j)}$.

Def: Softmax: $\text{softmax}(x) = \frac{\exp(x_i)}{\sum_{j=1}^d \exp(x_j)}$.

Def: Softmax: $\text{softmax}(x) = \frac{\exp(x_i)}{\sum_{j=1}^d \exp(x_j)}$.

Def: Softmax: $\text{softmax}(x) = \frac{\exp(x_i)}{\sum_{j=1}^d \exp(x_j)}$.

Def: Softmax: $\text{softmax}(x) = \frac{\exp(x_i)}{\sum_{j=1}^d \exp(x_j)}$.

Def: Softmax: $\text{softmax}(x) = \frac{\exp(x_i)}{\sum_{j=1}^d \exp(x_j)}$.

Def: Softmax: $\text{softmax}(x) = \frac{\exp(x_i)}{\sum_{j=1}^d \exp(x_j)}$.

Def: Softmax: $\text{softmax}(x) = \frac{\exp(x_i)}{\sum_{j=1}^d \exp(x_j)}$.

Def: Softmax: $\text{softmax}(x) = \frac{\exp(x_i)}{\sum_{j=1}^d \exp(x_j)}$.

Def: Softmax: $\text{softmax}(x) = \frac{\exp(x_i)}{\sum_{j=1}^d \exp(x_j)}$.

Def: Softmax: $\text{softmax}(x) = \frac{\exp(x_i)}{\sum_{j=1}^d \exp(x_j)}$.

Def: Softmax: $\text{softmax}(x) = \frac{\exp(x_i)}{\sum_{j=1}^d \exp(x_j)}$.

Def: Softmax: $\text{softmax}(x) = \frac{\exp(x_i)}{\sum_{j=1}^d \exp(x_j)}$.

Def: Softmax: $\text{softmax}(x) = \frac{\exp(x_i)}{\sum_{j=1}^d \exp(x_j)}$.

Def: Softmax: $\text{softmax}(x) = \frac{\exp(x_i)}{\sum_{j=1}^d \exp(x_j)}$.

Def: Softmax: $\text{softmax}(x) = \frac{\exp(x_i)}{\sum_{j=1}^d \exp(x_j)}$.

Def: Softmax: $\text{softmax}(x) = \frac{\exp(x_i)}{\sum_{j=1}^d \exp(x_j)}$.

Def: Softmax: $\text{softmax}(x) = \frac{\exp(x_i)}{\sum_{j=1}^d \exp(x_j)}$.

Def: Softmax: $\text{softmax}(x) = \frac{\exp(x_i)}{\sum_{j=1}^d \exp(x_j)}$.

Def: Softmax: $\text{softmax}(x) = \frac{\exp(x_i)}{\sum_{j=1}^d \exp(x_j)}$.

Def: Softmax: $\text{softmax}(x) = \frac{\exp(x_i)}{\sum_{j=1}^d \exp(x_j)}$.

Def: Softmax: $\text{softmax}(x) = \frac{\exp(x_i)}{\sum_{j=1}^d \exp(x_j)}$.

Def: Softmax: $\text{softmax}(x) = \frac{\exp(x_i)}{\sum_{j=1}^d \exp(x_j)}$.

Def: Softmax: $\text{softmax}(x) = \frac{\exp(x_i)}{\sum_{j=1}^d \exp(x_j)}$.

Def: Softmax: $\text{softmax}(x) = \frac{\exp(x_i)}{\sum_{j=1}^d \exp(x_j)}$.

Def: Softmax: $\text{softmax}(x) = \frac{\exp(x_i)}{\sum_{j=1}^d \exp(x_j)}$.

Def: Softmax: $\text{softmax}(x) = \frac{\exp(x_i)}{\sum_{j=1}^d \exp(x_j)}$.

Def: Softmax: $\text{softmax}(x) = \frac{\exp(x_i)}{\sum_{j=1}^d \exp(x_j)}$.

Def: Softmax: $\text{softmax}(x) = \frac{\exp(x_i)}{\sum_{j=1}^d \exp(x_j)}$.

Def: Softmax: $\text{softmax}(x) = \frac{\exp(x_i)}{\sum_{j=1}^d \exp(x_j)}$.

Def: Softmax: $\text{softmax}(x) = \frac{\exp(x_i)}{\sum_{j=1}^d \exp(x_j)}$.

Def: Softmax: $\text{softmax}(x) = \frac{\exp(x_i)}{\sum_{j=1}^d \exp(x_j)}$.

Def: Softmax: $\text{softmax}(x) = \frac{\exp(x_i)}{\sum_{j=1}^d \exp(x_j)}$.

Def: Softmax: $\text{softmax}(x) = \frac{\exp(x_i)}{\sum_{j=1}^d \exp(x_j)}$.

3. Finite State LMs

3.1 Weighted FSA

Def: A FSA is a 5-tuple $A = (\Sigma, Q, I, F, \delta)$ where:

- Σ is an alphabet, Q is a set of states (finite), $I \subseteq Q$ is the set of initial states, $F \subseteq Q$ is the set of final/accepting states, $\delta: Q \times \Sigma \rightarrow 2^Q$. Elements of δ called transitions.

Def: A FSA $A = (\Sigma, Q, I, F, \delta)$ is deterministic if:

- (i) it does not have any ϵ -transitions ($q \xrightarrow{\epsilon} q'$).
- (ii) $\forall q, a \in Q \times \Sigma$, $\exists$ at most one $q' \in Q$ s.t. $q \xrightarrow{a} q'$.

Otherwise, A is non-deterministic.

Def: Let $A = (\Sigma, Q, I, F, \delta)$ be a FSA. The language of A , $L(A) = \{x \in \Sigma^* | x \text{ is recognized by } A\}$.

Def: A language $L \subseteq \Sigma^*$ is regular, iff it can be recognized by an unweighted FSA $\exists A: L = L(A)$.

Def: A real-weighted FSA (WFSA) $A = (\Sigma, Q, \delta, \lambda, \rho)$.

Σ is a finite alphabet, Q is a finite set of states, $\delta: Q \times \Sigma \rightarrow 2^Q$ is a finite set of transitions, $\lambda: Q \rightarrow \mathbb{R}^+$, $\rho: Q \rightarrow \mathbb{R}^+$ weighting fct's.

Def: $\lambda: Q \rightarrow \mathbb{R}^+$, $\rho: Q \rightarrow \mathbb{R}^+$ weighting fct's.

Def: $\lambda: Q \rightarrow \mathbb{R}^+$, $\rho: Q \rightarrow \mathbb{R}^+$ weighting fct's.

Def: $\lambda: Q \rightarrow \mathbb{R}^+$, $\rho: Q \rightarrow \mathbb{R}^+$ weighting fct's.

Def: $\lambda: Q \rightarrow \mathbb{R}^+$, $\rho: Q \rightarrow \mathbb{R}^+$ weighting fct's.

Def: $\lambda: Q \rightarrow \mathbb{R}^+$, $\rho: Q \rightarrow \mathbb{R}^+$ weighting fct's.

Def: $\lambda: Q \rightarrow \mathbb{R}^+$, $\rho: Q \rightarrow \mathbb{R}^+$ weighting fct's.

Def: $\lambda: Q \rightarrow \mathbb{R}^+$, $\rho: Q \rightarrow \mathbb{R}^+$ weighting fct's.

Def: $\lambda: Q \rightarrow \mathbb{R}^+$, $\rho: Q \rightarrow \mathbb{R}^+$ weighting fct's.

Def: $\lambda: Q \rightarrow \mathbb{R}^+$, $\rho: Q \rightarrow \mathbb{R}^+$ weighting fct's.

Def: $\lambda: Q \rightarrow \mathbb{R}^+$, $\rho: Q \rightarrow \mathbb{R}^+$ weighting fct's.

Def: $\lambda: Q \rightarrow \mathbb{R}^+$, $\rho: Q \rightarrow \mathbb{R}^+$ weighting fct's.

Def: $\lambda: Q \rightarrow \mathbb{R}^+$, $\rho: Q \rightarrow \mathbb{R}^+$ weighting fct's.

Def: $\lambda: Q \rightarrow \mathbb{R}^+$, $\rho: Q \rightarrow \mathbb{R}^+$ weighting fct's.

Def: $\lambda: Q \rightarrow \mathbb{R}^+$, $\rho: Q \rightarrow \mathbb{R}^+$ weighting fct's.

Def: $\lambda: Q \rightarrow \mathbb{R}^+$, $\rho: Q \rightarrow \mathbb{R}^+$ weighting fct's.

Def: $\lambda: Q \rightarrow \mathbb{R}^+$, $\rho: Q \rightarrow \mathbb{R}^+$ weighting fct's.

Def: $\lambda: Q \rightarrow \mathbb{R}^+$, $\rho: Q \rightarrow \mathbb{R}^+$ weighting fct's.

Def: $\lambda: Q \rightarrow \mathbb{R}^+$, $\rho: Q \rightarrow \mathbb{R}^+$ weighting fct's.

Def: $\lambda: Q \rightarrow \mathbb{R}^+$, $\rho: Q \rightarrow \mathbb{R}^+$ weighting fct's.

Def: $\lambda: Q \rightarrow \mathbb{R}^+$, $\rho: Q \rightarrow \mathbb{R}^+$ weighting fct's.

Def: $\lambda: Q \rightarrow \mathbb{R}^+$, $\rho: Q \rightarrow \mathbb{R}^+$ weighting fct's.

Def: $\lambda: Q \rightarrow \mathbb{R}^+$, $\rho: Q \rightarrow \mathbb{R}^+$ weighting fct's.

Def: $\lambda: Q \rightarrow \mathbb{R}^+$, $\rho: Q \rightarrow \mathbb{R}^+$ weighting fct's.

Def: $\lambda: Q \rightarrow \mathbb{R}^+$, $\rho: Q \rightarrow \mathbb{R}^+$ weighting fct's.

Def: $\lambda: Q \rightarrow \mathbb{R}^+$, $\rho: Q \rightarrow \mathbb{R}^+$ weighting fct's.

Def: $\lambda: Q \rightarrow \mathbb{R}^+$, $\rho: Q \rightarrow \mathbb{R}^+$ weighting fct's.

Def: $\lambda: Q \rightarrow \mathbb{R}^+$, $\rho: Q \rightarrow \mathbb{R}^+$ weighting fct's.

Def: $\lambda: Q \rightarrow \mathbb{R}^+$, $\rho: Q \rightarrow \mathbb{R}^+$ weighting fct's.

Def: $\lambda: Q \rightarrow \mathbb{R}^+$, $\rho: Q \rightarrow \mathbb{R}^+$ weighting fct's.

Def: $\lambda: Q \rightarrow \mathbb{R}^+$, $\rho: Q \rightarrow \mathbb{R}^+$ weighting fct's.

Def: $\lambda: Q \rightarrow \mathbb{R}^+$, $\rho: Q \rightarrow \mathbb{R}^+$ weighting fct's.

Def: $\lambda: Q \rightarrow \mathbb{R}^+$, $\rho: Q \rightarrow \mathbb{R}^+$ weighting fct's.

Def: $\lambda: Q \rightarrow \mathbb{R}^+$, $\rho: Q \rightarrow \mathbb{R}^+$ weighting fct's.

Def: $\lambda: Q \rightarrow \mathbb{R}^+$, $\rho: Q \rightarrow \mathbb{R}^+$ weighting fct's.

Def: $\lambda: Q \rightarrow \mathbb{R}^+$, $\rho: Q \rightarrow \mathbb{R}^+$ weighting fct's.

Def: $\lambda: Q \rightarrow \mathbb{R}^+$, $\rho: Q \rightarrow \mathbb{R}^+$ weighting fct's.

Def: $\lambda: Q \rightarrow \mathbb{R}^+$, $\rho: Q \rightarrow \mathbb{R}^+$ weighting fct's.

Def: $\lambda: Q \rightarrow \mathbb{R}^+$, $\rho: Q \rightarrow \mathbb{R}^+$ weighting fct's.

Def: $\lambda: Q \rightarrow \mathbb{R}^+$, $\rho: Q \rightarrow \mathbb{R}^+$ weighting fct's.

4. Pushdown LMs

4.1 Human Lang. is not finite state

Def: Human Lang. is not finite state (FLMS).

Def: Human Lang. is not finite state (FLMS).

Def: Human Lang. is not finite state (FLMS).

Def: Human Lang. is not finite state (FLMS).

Def: Human Lang. is not finite state (FLMS).

Def: Human Lang. is not finite state (FLMS).

Def: Human Lang. is not finite state (FLMS).

Def: Human Lang. is not finite state (FLMS).

Def: Human Lang. is not finite state (FLMS).

Def: Human Lang. is not finite state (FLMS).

Def: Human Lang. is not finite state (FLMS).

Def: Human Lang. is not finite state (FLMS).

Def: Human Lang. is not finite state (FLMS).

Def: Human Lang. is not finite state (FLMS).

Def: Human Lang. is not finite state (FLMS).

Def: Human Lang. is not finite state (FLMS).

Def: Human Lang. is not finite state (FLMS).

Def: Human Lang. is not finite state (FLMS).

Def: Human Lang. is not finite state (FLMS).

Def: Human Lang. is not finite state (FLMS).

Def: Human Lang. is not finite state (FLMS).

Def: Human Lang. is not finite state (FLMS).

Def: Human Lang. is not finite state (FLMS).

Def: Human Lang. is not finite state (FLMS).

Def: Human Lang. is not finite state (FLMS).

Def: Human Lang. is not finite state (FLMS).

Def: Human Lang. is not finite state (FLMS).

Def: Human Lang. is not finite state (FLMS).

Def: Human Lang. is not finite state (FLMS).

Def: Human Lang. is not finite state (FLMS).

Def: Human Lang. is not finite state (FLMS).

Def: Human Lang. is not finite state (FLMS).

Def: Human Lang. is not finite state (FLMS).

Def: Human Lang. is not finite state (FLMS).

Def: Human Lang. is not finite state (FLMS).

Def: Human Lang. is not finite state (FLMS).

Def: Human Lang. is not finite state (FLMS).

Def: Human Lang. is not finite state (FLMS).

Def: Human Lang. is not finite state (FLMS).

Def: Human Lang. is not finite state (FLMS).

Def: Human Lang. is not finite state (FLMS).

Def: Human Lang. is not finite state (FLMS).

Def: Human Lang. is not finite state (FLMS).

Def: Human Lang. is not finite state (FLMS).

Def: Human Lang. is not finite state (FLMS).

Def: Human Lang. is not finite state (FLMS).

Def: Human Lang. is not finite state (FLMS).

Def: Human Lang. is not finite state (FLMS).

5. Neural Network LMs

5.1 RNNs

Def: Let Σ be an alphabet. A deterministic real valued RNN R is a 4-tuple (Σ, O, f, h_0) where:

- Σ : alphabet of input symbols;
- O : dim of R ; $R: \mathbb{R}^d \times \mathbb{R}^d \rightarrow \mathbb{R}^d$ dynamics map;
- $h_0 \in \mathbb{R}^d$ is the initial state.

Def: Let R be a RNN. A recurrent neural encoding fct. enc_R is a repr. fct.: $\text{enc}_R(x) = f(\text{enc}_R(x_{1:n}), x_n) \in \mathbb{R}^d$.

Also called hidden state: $h_n = \text{enc}_R(x_{1:n}) = f(h_{n-1}, x_n)$.

Def: Let R be RNN & $E \in \mathbb{R}^{d \times d}$ a symbol repr. mat.

A O-dim recurrent neural seq. model over alph. Σ is a tuple (Σ, O, f, E, h_0) def. the seq. model.

Def: Saturated Symbol: $R(x) = f(x, x)$ if $x \in \Sigma$, $f(x, x) = 1$ if $x \in \Sigma$.

Thm: RNN seq. mod. is with saturated O-dim fct. are Turing complete.

Rem: A real-time RNN is not Turing complete.

Thm: RNNs can recognize def. context-free languages.

5.2 Transformer

Def: A transformer network T is a tuple $(\Sigma, O, \text{enc}_T)$ where:

- Σ : alphabet of input symbols, $O = \dim(T)$;
- enc_T : transformer enc. fct.

Def: hidden state $h_n \in \mathbb{R}^d$, $h_n = \text{enc}_T(x_{1:n})$.

Def: Let T be a transf. netw. & $E \in \mathbb{R}^{d \times d}$ a symbol repr. mat.

A O-dim transf. seq. mod. over the alphabet Σ is a tuple $(\Sigma, O, \text{enc}_T, E)$ def. the seq. model.

$\text{enc}_T(x) = \text{softmax}(\frac{E \cdot \text{enc}_T(x)}{\sum_{y \in \Sigma} E \cdot \text{enc}_T(y)})$.

Def: Let $f: \mathbb{R}^d \rightarrow \mathbb{R}^d$ be a scoring fct. & f_{proj} a projection fct. Furthermore, let $q \in \mathbb{R}^d$, $k \in \mathbb{R}^d$, $v \in \mathbb{R}^d$, $E \in \mathbb{R}^{d \times d}$.

$q = \text{enc}_T(x)$, $k = \text{enc}_T(y)$, $v = \text{enc}_T(z)$.

Def: Soft Attention computed with f_{proj} & f_{scoring} .

Def: Averaging hard attention is att. mech. w/ f_{proj} & $f_{\text{sc$

